ROS 2 Konu Anlatımı

Sıfırdan Başlayan Biri İçin — Kara Muhafız Projesi Üzerinden Anlatım (güncelleme: 2026-10-04, Gün 4-5 eklendi)

Bu doküman, ROS 2'yi hiç bilmeyen birine, Kara Muhafız projesinde şu ana kadar gerçekten yazdığımız ve çalıştırdığımız kodlar üzerinden ROS 2'nin temel kavramlarını anlatır. Her bölüm önce kavramı günlük hayattan bir benzetmeyle açıklar, sonra bizim projemizdeki gerçek kod örneğini gösterir. Amaç, ezber değil — "bu satır neden böyle yazıldı, ne işe yarıyor" sorusuna her zaman cevap verebilmek.

İçindekiler

1. ROS 2 Nedir, Neden Var?
2. Node — En Küçük Yapı Taşı
3. Topic, Publisher, Subscriber
4. Paket (Package) ve Workspace Yapısı
5. Launch Dosyaları — Her Şeyi Tek Komutla Başlatmak
6. Robotu Tanımlamak: URDF ve Xacro
7. TF — Koordinat Çerçeveleri ve Dönüşümler
8. Gazebo Simülasyonu ve Plugin Mimarisi
9. ros2_control — Donanımı (veya Simülasyonu) Kontrol Etmek
10. Ackermann Direksiyon Kinematiği
11. Parametreler ve YAML
12. Faydalı Komutlar — Hızlı Başvuru
13. Büyük Resim: Gün 1-3 Sistemi
14. Kendi Node'unu Yazmak (Python / rclpy)
15. Launch Argümanları ve Modüler Launch Dosyaları
16. Klavye Girdisinin Sınırları ve Watchdog Deseni

1. ROS 2 Nedir, Neden Var?

ROS 2 (Robot Operating System 2), bir işletim sistemi DEĞİLDİR — Ubuntu'nun üzerinde çalışan bir yazılım çatısıdır (framework). Bir robotta onlarca farklı program aynı anda çalışır: kamera verisini okuyan, GPS'i okuyan, tekerlekleri döndüren, rota planlayan, engelden kaçan programlar... ROS 2'nin tek işi, bu programların BİRBİRİYLE KONUŞMASINI standart ve güvenilir bir şekilde sağlamak.

> *Günlük hayattan benzetme: Bir restoran mutfağını düşün: aşçı, garson, bulaşıkçı, kasiyer hepsi ayrı işler yapar ama ortak bir "sipariş sistemi" üzerinden haberleşirler (mutfak ekranına sipariş düşer, garson alır, hazır olunca haber verilir). ROS 2 işte bu "sipariş sistemi"dir — her program (aşçı, garson) kendi işine odaklanır, ROS 2 aralarındaki haberleşmeyi standartlaştırır.*

Bizim projemizde onlarca ayrı program aynı anda çalışıyor: gzserver (fizik motoru), robot_state_publisher (TF yayıncısı), controller_manager (tekerlek kontrolcüsü), manual_drive_node (bizim yazdığımız sürüş adaptörü)... Hepsi ROS 2 sayesinde birbirinden habersiz şekilde yazılabiliyor ama birbiriyle konuşabiliyor.

2. Node — En Küçük Yapı Taşı

Bir node, tek bir işi yapan, çalışan bir programdır (bir süreç/process). ROS 2'de "büyük bir program" yazmak yerine, her biri küçük bir işi yapan birçok node yazılır.

> *Günlük hayattan benzetme: Mutfaktaki her çalışan (aşçı, garson) ayrı bir "node"dur. Aşçı sadece yemek pişirir, garson sadece sipariş taşır — birbirinin işine karışmaz.*

Bizim sistemimizde şu anda çalışan node'lardan bazıları:

- robot_state_publisher — URDF'den TF (koordinat) ağacını üretir
- controller_manager (gazebo_ros2_control eklentisi içinde) — tekerlek controller'larını yönetir
- joint_state_broadcaster — eklem pozisyon/hızlarını /joint_states'e yayınlar
- ackermann_steering_controller — Twist komutunu teker hareketine çevirir
- manual_drive_node — bizim yazdığımız, teleop'tan gelen komutu güvenlik limitleriyle controller'a aktaran node
- keyboard_teleop_node / global_keyboard_teleop_node — bizim yazdığımız klavye teleop node'ları

```
$ ros2 node list
/robot_state_publisher
/controller_manager
/joint_state_broadcaster
/ackermann_steering_controller
/manual_drive_node
...

$ ros2 node info /robot_state_publisher
  # bu node'un hangi topic'leri yayınladığını/dinlediğini gösterir
```

3. Topic, Publisher, Subscriber

Node'lar birbirine DOĞRUDAN bağlanmaz (biri diğerinin adresini bilmez). Bunun yerine, isimli "kanallara" (topic) veri yayınlarlar (publish) veya o kanalı dinlerler (subscribe). Bir topic'i kim dinliyor, publisher hiç bilmez — bu, sistemi esnek yapar.

> *Günlük hayattan benzetme: Bir radyo istasyonu (örn. "Kanal 5") düşün. Spiker (publisher) konuşur, o kanalı açmış herkes (subscriber) duyar. Spiker kimin dinlediğini bilmez, umursamaz da — dinleyici sayısı 0 da olsa 1000 de olsa aynı şekilde yayın yapar.*

Her topic'in bir mesaj tipi vardır — o kanalda hangi "formatta" veri aktığını belirler. Örneğin /joint_states topic'i sensor_msgs/msg/JointState tipinde mesajlar taşır (eklem isimleri, açıları, hızları).

```
$ ros2 topic list
```

```
/joint_states
/tf
/cmd_vel
/ackermann_steering_controller/reference_unstamped
/ackermann_steering_controller/odometry
...

$ ros2 topic echo /joint_states --once   # bir mesajı goster
$ ros2 topic hz /joint_states            # saniyede kac mesaj geliyor
$ ros2 topic info /joint_states -v       # kac publisher/subscriber var, QoS nedir
```

Gün 4-5'te kurduğumuz zincir tam olarak bu prensibe dayanıyor: keyboard_teleop_node /cmd_vel'e YAYINLAR, manual_drive_node /cmd_vel'i DİNLER ve işleyip /ackermann_steering_controller/reference_unstamped'e YAYINLAR, ackermann_steering_controller da onu DİNLER. Üç node birbirini hiç tanımıyor, sadece ortak topic isimlerini biliyorlar.

4. Paket (Package) ve Workspace Yapısı

ROS 2'de kod tek tek dosyalar halinde değil, paket (package) adı verilen klasörler içinde organize edilir. Her paketin bir kimliği (package.xml) ve derleme talimatı (CMakeLists.txt veya setup.py) vardır.

> *Günlük hayattan benzetme: Bir paket, bir mutfak bölümü gibi düşünülebilir (örn. "tatlı bölümü"). İçinde kendi malzemeleri (dosyaları) ve kendi tarifleri (kodu) vardır; diğer bölümlerle (paketlerle) sadece belirlenen "sipariş" arayüzü (topic/service) üzerinden konuşur.*

Bizim projemizde üç paket var:

- karamuhafiz_base (ament_cmake) — verilen (değiştirilemez) araç modeli, mesh'ler, parkur world dosyası. Yalnızca VERİ içerir.
- karamuhafiz_bringup (ament_cmake) — bizim yazdığımız launch dosyaları ve controller yapılandırmaları.
- karamuhafiz_control (ament_python) — bizim yazdığımız gerçek Python node'ları (manuel sürüş, teleop).

İki paket tipi arasındaki fark: ament_cmake paketleri CMakeLists.txt ile derlenir/dosya kopyalar (C++ veya saf veri/launch paketleri için); ament_python paketleri ise setup.py + setup.cfg kullanır ve Python modüllerini "paket" olarak kurar. İkisi de aynı workspace içinde bir arada, sorunsuz çalışabilir.

Birden fazla paketin bir arada derlendiği üst klasöre workspace denir:

```
karamuhafiz_ros2_2025_arac_parkur/      <- workspace koku
  src/                                  <- kaynak kod (biz buraya yaziyoruz)
    karamuhafiz_base/      (ament_cmake)
    karamuhafiz_bringup/   (ament_cmake)
    karamuhafiz_control/   (ament_python)
  build/                                <- colcon'un ara derleme dosyalari
  install/                              <- CALISTIRILABILIR son hal buraya kopyalanir
  log/
```

colcon build --symlink-install tüm paketleri derler. --symlink-install, dosyaları kopyalamak yerine kısayol oluşturur — src/'deki bir launch/config/Python dosyasını değiştirdiğimizde yeniden build etmeye GEREK KALMAZ (saf Python dosyaları ve YAML/launch dosyaları için; yeni bir dosya/paket eklendiğinde veya setup.py değiştiğinde yine de build gerekir).

5. Launch Dosyaları — Her Şeyi Tek Komutla Başlatmak

Simülasyonumuzu çalıştırmak için 6-7 ayrı programı (gzserver, gzclient, robot_state_publisher, spawn_entity, iki controller spawner'ı, manuel sürüş node'u) belirli bir sırayla ve doğru parametrelerle başlatmamız gerekiyor. Bunu her seferinde elle tek tek yapmak yerine, hepsini TEK

bir Python dosyasında tarif ederiz — buna launch dosyası denir.

> *Günlük hayattan benzetme: Bir orkestra şefinin notası gibi düşün: "önce yaylılar başlasın, 3. saniyede üflemeliler girsin..." Launch dosyası da ROS 2'ye hangi programın ne zaman, hangi ayarlarla başlayacağını söyler.*

Bizim sim_bringup.launch.py dosyamızın iskeleti:

```
def generate_launch_description():
    # ... dosya yollarini bul, xacro'yu URDF'e cevir ...

    return LaunchDescription([
        set_model_path,              # Gazebo'ya mesh'leri nerede bulacagini soyler
        gzserver,                    # fizik motoru
        gzclient,                    # gorsel pencere
        robot_state_publisher,       # TF yayincisi
        spawn_entity,                # araci Gazebo'ya "dogurur"
        joint_state_broadcaster_spawner,
        ackermann_controller_spawner,
    ])
```

İki tür başlatma elemanı kullandık: Node(...) gerçek bir ROS 2 node'u başlatır (package + executable belirtilir, ROS 2'nin kendi altyapısını kullanır). ExecuteProcess(...) ise düz bir komut satırı programını çalıştırır (gzserver bir ROS 2 node'u değil, düz bir Gazebo programı olduğu için bunu kullandık).

```
$ ros2 launch karamuhafiz_bringup manual_drive.launch.py
```

6. Robotu Tanımlamak: URDF ve Xacro

URDF (Unified Robot Description Format), bir robotun fiziksel yapısını XML ile tarif eden standart format. İki temel öğesi vardır:

- link — robotun katı bir parçası (gövde, tekerlek, kamera kutusu...). Görsel (visual), çarpışma (collision) ve kütle/atalet (inertial) bilgisi taşır.
- joint — iki link'i birbirine bağlayan eklem (sabit, dönel, kaymalı...). Hangi link'in "parent", hangisinin "child" olduğunu ve hareket eksenini tanımlar.

karamuhafiz.urdf.xacro'da 30 link ve 39 joint var — gövde, 4 tekerlek, 4 süspansiyon, 2 direksiyon mafsalı, kameralar, sonar sensörleri, taret/namlu...

```
<link name="base_footprint"/>
<joint name="base_footprint_joint" type="fixed">
  <parent link="base_footprint"/><child link="base_link"/>
  <origin xyz="0 0 0.2875" rpy="0 0 0"/>
</joint>
```

Bu örnekte base_footprint (zemin referans noktası) ile base_link (araç gövdesi) arasında SABİT (fixed) bir eklem var — base_link, base_footprint'e göre 0.2875 m yukarıda duruyor (teker yarıçapı + yer boşluğu).

Xacro, URDF'in "programlanabilir" halidir — saf URDF'de her şeyi elle tek tek yazman gerekir (4 teker için aynı XML'i 4 kez kopyala-yapıştır gibi). Xacro'da MAKRO yazıp tekrar tekrar çağırabilirsin:

```
<xacro:macro name="km2_wheel_link" params="name mass ixx iyy izz">
  <!-- bir tekerin tüm tanımı burada -->
</xacro:macro>

<xacro:km2_wheel_link name="front_l" mass="4.0" .../>
<xacro:km2_wheel_link name="front_r" mass="4.0" .../>
<xacro:km2_wheel_link name="rear_l"  mass="4.2" .../>
<xacro:km2_wheel_link name="rear_r"  mass="4.2" .../>
```

Ayrıca xacro, dışarıdan parametre almayı da sağlar — <xacro:arg name="controllers_yaml" default=""/> satırı, bizim launch dosyamızdan "bu xacro'yu işlerken controllers_yaml diye bir değer de geçeceğim" demek. Komut satırında xacro çalıştırırken bu şekilde kullanılır:

```
xacro karamuhafiz.urdf.xacro controllers_yaml:=/tam/yol/controllers.yaml
```

Xacro dosyası, xacro programıyla İŞLENDİĞİNDE düz bir URDF/XML metnine dönüşür — Gazebo ve robot_state_publisher sadece bu SONUÇ metni anlar, xacro söz dizimini (macro, arg) bilmez.

7. TF — Koordinat Çerçeveleri ve Dönüşümler

Bir robotta her parçanın kendi "bakış açısı" (koordinat çerçevesi/frame) vardır: kamera kendi merkezinden "öndeki engel 2 metre ileride" der, ama aracın GPS konumuna göre bu engelin yeri nedir? İşte TF (transform), tüm bu çerçeveler arasındaki konum/yön ilişkisini sürekli günceller ve soru sorulduğunda cevap verir.

> *Günlük hayattan benzetme: Bir gemideki "köprü, makine dairesi, güverte" gibi farklı referans noktalarını düşün. "Köprüden 10 metre batıda" demek herkese bir şey ifade etmez ama "enlem/boylam X,Y" dersen herkes anlar. TF, robotun içindeki tüm bu yerel referansları ortak bir "harita diline" çevirir.*

TF bir AĞAÇ yapısındadır — her frame'in bir ebeveyni vardır (URDF'deki joint'lerin parent/child ilişkisiyle BİREBİR aynı mantık). Bizim sistemimizde (hedeflenen) zincir:

```
map -> odom -> base_footprint -> base_link -> [tekerlekler, kameralar, sensorler, ...]
```

robot_state_publisher node'u, URDF'deki SABİT (fixed) eklemleri ve joint_state_broadcaster'dan gelen HAREKETLİ eklem açılarını (/joint_states) birleştirip bu TF ağacını sürekli yayınlar.

odom -> base_footprint kısmını ise ackermann_steering_controller yayınlıyor (enable_odom_tf: true ayarıyla) — tekerlek hareketlerinden aracın ne kadar ilerlediğini hesaplayıp (odometri) bunu TF'e ekliyor.

8. Gazebo Simülasyonu ve Plugin Mimarisi

Gazebo, fiziği (yerçekimi, sürtünme, çarpışma) gerçekçi şekilde hesaplayan bir simülatördür. Kendi başına ROS 2 bilmez — ikisini birbirine bağlayan köprüye plugin denir.

- gzserver — fizik hesaplamalarını yapan arka plan süreci (GUI yok)
- gzclient — gzserver'a bağlanıp 3B görsel pencereyi açan süreç
- libgazebo_ros_factory.so — /spawn_entity servisini sağlar (modelleri sonradan simülasyona ekleyebilmek için)
- libgazebo_ros2_control.so — Gazebo fiziğini ros2_control'e bağlayan köprü
- libgazebo_ros_camera.so / libgazebo_ros_ray_sensor.so — kamera/sonar gibi sensörlerin verisini ROS 2 topic'lerine çeviren eklentiler

world dosyası (teknofest_parkur.world), parkurun statik modellerini (bariyer, koni, tabela, rampa) ve fizik motoru ayarlarını (yerçekimi, ODE çözücü iterasyon sayısı) tanımlar. Bu dosyaya hiç dokunmadık — şartname gereği değiştirilemez.

spawn_entity.py, bir modeli ÇALIŞAN simülasyona sonradan ekler. -x/-y/-z ile konum, -R/-P/-Y ile başlangıç yönelimi (roll/pitch/yaw, radyan) verilebilir. Gün 4-5'te aracın yanlış yönde doğduğunu fark edince, -Y 3.14159265 (180°) vererek doğru yönde doğmasını sağladık — bu, verilen world/xacro dosyalarına dokunmadan, SADECE bizim launch dosyamızdaki bir parametre.

9. ros2_control — Donanımı (veya Simülasyonu) Kontrol Etmek

ros2_control, gerçek bir robottaki motor sürücülerini VEYA (bizim durumumuzda) Gazebo'daki simüle eklemleri AYNI arayüzle kontrol etmeyi sağlayan bir çatıdır. Buradaki en önemli fikir: üstteki kontrol kodu ("tekerleği şu hızda döndür") gerçek donanımda mı yoksa simülasyonda mı çalıştığını bilmek ZORUNDA DEĞİLDİR.

Günlük hayattan benzetme: Bir arabanın direksiyon simülatörü (sürücü kursu) ile gerçek bir arabayı düşün: ikisinde de aynı direksiyon simidi, aynı pedallar var. Sürücü aynı şekilde kullanır; arkada "gerçek motor mu dönüyor yoksa ekrandaki animasyon mu" fark etmez. ros2_control'deki hardware interface, işte bu "ortak direksiyon simidi" katmanıdır.

Mimarideki üç ana parça:

- Hardware Interface — gerçek donanım veya simülasyonla konuşan en alt katman. Bizde bu, gazebo_ros2_control eklentisinin sağladığı "GazeboSystem" (xacro'daki <ros2_control> bloğunda tanımlı).
- Controller Manager — hangi controller'ların yüklü/aktif olduğunu yönetir, belirli bir frekansta (bizde 100 Hz) hepsini günceller.
- Controller — asıl mantığı yürüten parça. İki alt tipi var: broadcaster (sadece durum OKUR, komut vermez — örn. joint_state_broadcaster) ve controller (komut ALIR, eklemlere YAZAR — örn. ackermann_steering_controller).

Her eklemin (joint) hangi arayüzleri desteklediği xacro'da tanımlanır:

```
<joint name="rear_l_joint">
  <command_interface name="velocity"/>   <!-- KOMUT alabilir -->
  <state_interface name="position"/>     <!-- durumu OKUNABILIR -->
  <state_interface name="velocity"/>
</joint>
```

rear_l_joint (arka sol teker) bir velocity command_interface'ine sahip — yani bir controller buraya "şu hızda dön" yazabilir. front_l_steer_joint ise position command_interface'ine sahip — "şu açıya dön" komutu alır. ackermann_steering_controller tam olarak bu ikisini kullanıyor: arka tekerlere HIZ, ön direksiyon mafsallarına POZİSYON komutu gönderiyor.

```
$ ros2 control list_controllers
joint_state_broadcaster       joint_state_broadcaster/JointStateBroadcaster   active
ackermann_steering_controller ackermann_steering_controller/...               active
```

10. Ackermann Direksiyon Kinematiği

Bir arabanın dört tekerleği virajda AYNI açıyla dönmez — iç taraftaki teker dış tarafa göre daha keskin döner. Çünkü virajda her teker farklı yarıçaplı bir çember üzerinde hareket eder, ve tüm tekerlerin merkezleri AYNI noktada kesişmelidir. Bu geometrik kurala Ackermann direksiyon prensibi denir.

Günlük hayattan benzetme: Bir grup insanın el ele tutuşup daire şeklinde dönmesini düşün — dıştaki kişi içteki kişiden daha HIZLI ve daha GENİŞ bir çemberde yürümek zorundadır, yoksa el ele kopar. Arabanın iç ve dış tekerleri de aynı sebeple farklı açı/hızda hareket etmelidir.

ackermann_steering_controller, bizim gönderdiğimiz basit "geometry_msgs/Twist" komutunu (sadece linear.x = ileri hız, angular.z = dönüş hızı) alıp, aracın gerçek ölçülerini (wheelbase = dingil mesafesi, track = iz genişliği) kullanarak HER bir tekerleğin doğru açı/hızını hesaplar — biz bu karmaşık geometriyle hiç uğraşmıyoruz, controller bizim için yapıyor. Aracımız RWD (Rear-Wheel Drive, arkadan itişli): arka tekerler hız komutu alır (traksiyon), ön tekerler sadece pozisyon komutu alır (direksiyon) ve serbestçe döner.

11. Parametreler ve YAML

ROS 2'de her node'un çalışma zamanında ayarlanabilen parametreleri vardır (örn. update_rate: 100). Bunları KOD İÇİNE sabit sayı olarak gömmek yerine, dışarıdan bir YAML dosyasıyla veya komut satırından vermek, aynı kodu farklı ayarlarla (farklı araç, farklı parkur) tekrar yazmadan kullanabilmemizi sağlar — görev şartnamesinin de özellikle istediği şey bu.

YAML'da ros__parameters anahtarı, parametrelerin HANGİ NODE'a ait olduğunu belirtir (üstündeki anahtar node adıdır):

```
<node_adi>:
  ros__parameters:
    parametre_1: deger
    parametre_2: [liste, elemanlari]
```

Dikkat: YAML, girintiye (indentation) duyarlıdır ve "iki nokta üst üste + boşluk" (:) kalıbını özel bir anlamda (anahtar-değer ayracı) yorumlar — Gün 2'deki en zorlu hatamızın kök sebebi tam olarak buydu (XML yorumlarındaki ": " kalıpları, bir YAML ayrıştırıcısını yanılttı). Bu, YAML'ın ne kadar "hassas" bir format olduğunun iyi bir örneği.

Kendi Python node'larımızda (manual_drive_node, keyboard_teleop_node) da aynı prensip: declare_parameter(...) ile "bu parametreyi dışarıdan alabilirim" deriz, gerçek değer YAML dosyasından (launch'ta parameters=[yaml_yolu] ile) gelir — max_linear_speed, command_timeout gibi hiçbir sayı Python kodunun içine sabit yazılmadı.

12. Faydalı Komutlar — Hızlı Başvuru

Komut	Ne İşe Yarar
`ros2 node list`	Çalışan tüm node'ları listeler
`ros2 topic list`	Tüm aktif topic'leri listeler
`ros2 topic echo <topic>`	Bir topic'e gelen mesajları canlı gösterir
`ros2 topic hz <topic>`	Bir topic'in yayın frekansını ölçer
`ros2 topic pub --rate N <topic> <tip> "{...}"`	Topic'e sürekli mesaj yayınlar
`ros2 control list_controllers`	Yüklü controller'ları ve durumlarını gösterir
`ros2 launch <paket> <dosya>`	Bir launch dosyasını çalıştırır
`ros2 run <paket> <calisirlabilir>`	Tek bir node'u doğrudan çalıştırır
`colcon build --symlink-install`	Workspace'i derler (kısayollarla)
`ros2 pkg list | grep <isim>`	Bir paketin kurulu olup olmadığını kontrol eder

13. Büyük Resim: Gün 1-3 Sistemi

sim_bringup.launch.py çalıştırıldığında olanları sırayla özetleyelim:

- 1) xacro, karamuhafiz.urdf.xacro'yu gerçek XML'e çevirir.
- 2) gzserver, teknofest_parkur.world'ü yükler (parkur + fizik motoru).
- 3) robot_state_publisher başlar, URDF'i parametre olarak alır, TF yayınlamaya başlar.
- 4) spawn_entity, aracı Gazebo'ya (doğru yönelimle, -Y parametresiyle) ekler — bu sırada xacro içindeki <ros2_control> bloğu sayesinde gazebo_ros2_control eklentisi devreye girer ve controller_manager'ı başlatır.
- 5) controller_manager, controllers.yaml'ı okuyarak hangi controller'ların YÜKLENEBİLECEĞİNİ öğrenir (henüz aktif değiller).
- 6) İki spawner node'u sırayla controller_manager'a bağlanıp joint_state_broadcaster ve ackermann_steering_controller'ı yükler ve aktive eder.
- 7) Artık sistem hazır: /joint_states akıyor, TF ağacı tam, ve /ackermann_steering_controller/reference_unstamped topic'ine Twist mesajı göndererek aracı sürebiliyoruz.

14. Kendi Node'unu Yazmak (Python / rclpy)

Gün 4-5'te ilk defa HAZIR bir ROS 2 paketi değil, kendi node'umuzu (manual_drive_node) sıfırdan yazdık. rclpy (ROS 2'nin Python kütüphanesi) ile bir node'un iskeleti hep aynı kalıptadır:

```
import rclpy
from rclpy.node import Node

class ManualDriveNode(Node):
    def __init__(self):
        super().__init__('manual_drive_node')   # node'un ROS 2'deki ismi
        self.declare_parameter('max_linear_speed', 1.5)
        self._sub = self.create_subscription(Twist, '/cmd_vel', self._cb, 10)
        self._pub = self.create_publisher(Twist, '/output_topic', 10)
        self._timer = self.create_timer(0.05, self._tick)  # 20 Hz

def main():
    rclpy.init()
    node = ManualDriveNode()
    rclpy.spin(node)   # node'u canli tutar, callback'leri calistirir
    rclpy.shutdown()
```

Günlük hayattan benzetme: rclpy.spin(node) bir "nöbetçi" gibidir — node'u sürekli ayakta tutar ve yeni bir mesaj/zamanlayıcı tetiklendiğinde ilgili fonksiyonu (callback) çağırır. spin() çağrılmazsa node hiçbir şeye tepki vermez, hemen kapanır.

Dört temel yapı taşı kullandık:

- create_subscription(Tip, topic, callback, 10) — bir topic'i dinler; yeni mesaj geldiğinde callback fonksiyonunu çağırır. Son sayı (10), QoS kuyruk derinliği (kaç mesaj biriktirilecek).
- create_publisher(Tip, topic, 10) — bir topic'e mesaj yayınlamak için "kalem" verir (.publish(msg) ile kullanılır).
- create_timer(saniye, callback) — belirli aralıklarla (bizde 1/20 = 0.05s, yani 20 Hz) bir fonksiyonu otomatik çağırır — manual_drive_node'un watchdog/sürekli-yayın mantığının temeli.
- declare_parameter(isim, varsayılan) — bu node'un dışarıdan (YAML/launch'tan) ayarlanabilir bir parametresi olduğunu bildirir.

15. Launch Argümanları ve Modüler Launch Dosyaları

Bir launch dosyasının kendisini de "parametrik" (dışarıdan ayarlanabilir) yapmak için DeclareLaunchArgument kullanılır — bu, declare_parameter'ın launch dosyaları için karşılığıdır:

```
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration

spawn_yaw_arg = DeclareLaunchArgument(
    "spawn_yaw", default_value="3.14159265",
    description="Aracin baslangic yonelimi (yaw, radyan)."
)
spawn_yaw = LaunchConfiguration("spawn_yaw")
# ... arguments=[..., "-Y", spawn_yaw] icinde kullanilir
```

Böylece komut satırından değiştirilebilir: ros2 launch karamuhafiz_bringup sim_bringup.launch.py spawn_yaw:=0.0 — dosyayı hiç açmadan aracın yönünü deneyebiliriz.

Launch dosyalarını birbirinin İÇİNE almak için IncludeLaunchDescription kullanılır — kod tekrarı yapmadan bir launch dosyasını başka bir launch dosyasının üstüne inşa etmemizi sağlar:

```
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource

sim_bringup = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(bringup_share, "launch", "sim_bringup.launch.py")
    )
)
```

manual_drive.launch.py dosyamız tam olarak bunu yapıyor: sim_bringup.launch.py'yi (Gazebo + araç + controller'lar) İÇİNE alıp, üstüne manual_drive_node'u ekliyor. İleride otonomi için autonomous_drive.launch.py yazdığımızda, o da AYNI sim_bringup'ı include edecek — simülasyon kurulumu TEK bir yerde tanımlı kalacak.

16. Klavye Girdisinin Sınırları ve Watchdog Deseni

Gün 4-5'te ilginç bir donanım/işletim sistemi sınırıyla karşılaştık: bir terminal (TTY), bir tuşun ne zaman FİZİKSEL olarak bırakıldığını bilemez — yalnızca işletim sisteminin gönderdiği "tekrar" (key repeat) karakterlerini görür. Daha da önemlisi, standart X11 klavye tekrarı AYNI ANDA yalnızca TEK bir tuş için üretilir.

> *Günlük hayattan benzetme: Bir daktilo düşün: tuşa ne kadar süre bastığını değil, sadece HANGİ tuşa basıldığını "görür" (her basış bir karakter). Elini tuştan çeksen de çekmesen de daktilo bunu doğrudan bilemez — ancak yeni karakterler gelmeyi KESTİĞİNDE "galiba bıraktı" diye tahmin edebilir.*

Bu sınırı iki farklı şekilde ele aldık:

- keyboard_teleop_node (terminal tabanlı): "son ne zaman bu tuşun bir olayı geldi" zaman damgasını tutar; belirli bir süre (release_timeout) geçerse "bırakılmış" sayar. Gaz ekseni KALICI, direksiyon ekseni ANLIK tasarlandı — iki eksenin aynı anda çalışması gereken durumda (ileri + viraj) bu dengeyi sağlamak için.
- global_keyboard_teleop_node (pynput tabanlı): X11'in GERÇEK tuşa-bas/tuşu-bırak olaylarını (XRecord mekanizması üzerinden) doğrudan yakalar — tahmine gerek kalmaz, GERÇEK simültane çoklu tuş desteği sağlar, ayrıca pencere odağından bağımsız çalışır.

Her iki node da aynı watchdog desenini kullanıyor: bir create_timer ile SABİT frekansta (20 Hz) çalışan bir döngü, en son bilinen hız/dönüş değerini sürekli yayınlar (yeni klavye olayı olsun olmasın). Bu desen, robotikte çok yaygındır — "veri gelmezse ne olacağını" açıkça tanımlamak, sistemin ÖNGÖRÜLEBİLİR davranmasını sağlar. manual_drive_node'daki command_timeout da aynı deseni kullanıyor: /cmd_vel akışı kesilirse (0.3 saniyeden uzun süre yeni mesaj gelmezse) araç otomatik durur — bu, şartnamenin "komut kesildiğinde araç hareket etmemeli" kuralının somut uygulaması.

Bu doküman, Kara Muhafiz ROS 2 2025 Parkur Görevi kapsamında, Gün 1-5 arasında yazılan gerçek kod üzerinden hazırlanan, canlı güncellenen bir ROS 2 konu anlatımıdır. Günlük çalışma raporlarıyla (Gun1-2_Calisma_Raporu.pdf, Gun3_Calisma_Raporu.pdf, Gun4-5_Calisma_Raporu.pdf) birlikte okunması önerilir.